算法设计与分析

算法分析基础

方贤忠

宁波大学
计算机科学与技术系
fangxianzhong@nbu.edu.cn
https://xzfang.top

Spring & Summer, 2025



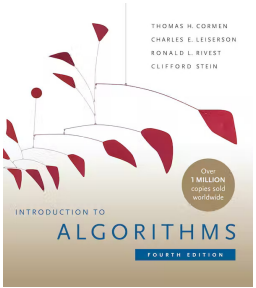
概要

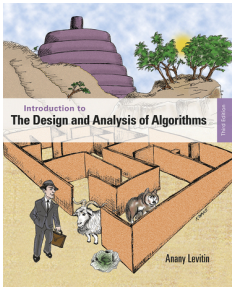
- 参考书
- 算法概述
- 基本运算、语句的运行时间分析
- 算法案例——查找
- 算法的渐近时间复杂性
- 渐近符号
- 算法与渐近时间复杂性的关系
- 递归算法分析
- 代入法
- 迭代法
- 主方法



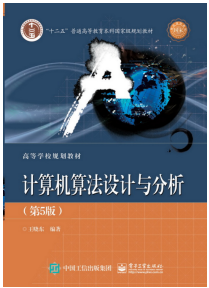
参考书

- 《算法导论》(Thomas H. Cormen, et al.)
- 《算法设计与分析基础》(Anany Levitin)
- 《算法》(Robert Sedgewick, et al.)
- 《计算机算法设计与分析》(王晓东)









成绩构成

- 平时成绩 (10%)
  - 课程思政 + 平时作业 + 课堂表现
- 实验 (40%)
  - 练习（每次系统开放一周）+ 堂练：每次 2-3 题, www.nbuoj.com
    - OJ 课程名称: 2025-107J29D-01-算法
  - 每个专题一次实验报告
    - 提交 Word 文档到学习平台 (学习通，study.nbu.edu.cn)
    - 实验模板见学习平台算法课程的资料文件夹
- 期末考试 (50%)



成绩构成

- 平时成绩 (10%)
  - 课程思政 + 平时作业 + 课堂表现
- 实验 (40%)
  - 练习（每次系统开放一周）+ 堂练：每次 2-3 题, www.nbuoj.com
    - OJ 课程名称: 2025-107J29D-01-算法
  - 每个专题一次实验报告
    - 提交 Word 文档到学习平台 (学习通，study.nbu.edu.cn)
    - 实验模板见学习平台算法课程的资料文件夹
- 期末考试 (50%)



成绩构成

- 平时成绩 (10%)
  - 课程思政 + 平时作业 + 课堂表现

关于实验报告:

- 首先每次 OJ 练习，线上提交完成（堂练测试限定 2 节课内提交）;
- 实验报告包括每个专题练习的所有题目: 代码 + 注释 + 说明分析 + 扩展 (自行做一些与实验同类型的题目);
- 每次 OJ 练习，OJ 排名与实验考核分相关.



算法

An **algorithm** is a step-by-step procedure for solving a problem in a *finite* amount of time.



算法

An **algorithm** is a step-by-step procedure for solving a problem in a *finite* amount of time.

算法是指解决问题的一种方法或一个过程.



算法

An **algorithm** is a step-by-step procedure for solving a problem in a *finite* amount of time.

算法是指解决问题的一种方法或一个过程.

算法是若干指令的有穷序列，满足性质：

1) 输入: 有外部提供的量作为算法的输入;
2) 输出: 算法产生至少一个量作为输出;
3) 确定性: 组成算法的每条指令清晰、无歧义;
4) 有限性: 每条指令的执行次数、执行时间有限.



算法

- 如何设计算法: 确定的、非确定的、哪类算法
- 如何表示算法: 描述 $\rightarrow$ 代码，伪代码
- 如何确认算法: 可行性（从算法角度看正确性）
- 如何分析算法: 空间、时间复杂性分析
- 如何测试算法: 软件工程（黑盒、白盒），使用公开的测试数据验证（从代码和输入输出角度看正确性）



基本运算、语句的运行时间分析

- 规则 (1) 赋值、比较、算术运算、逻辑运算、读写单个常量或单个变量等, 只需 1 个单位时间.
- 规则 (2) 条件语句"if (C) S_1; else S_2;"
  - 只需要 $T_c + \max(T_{s1}, T_{s2})$ 的时间;
  - T_c 是计算条件表达式 C 需要的时间;
  - T_{s1} 和 T_{s2} 分别是执行语句 S_1 和 S_2 需要的时间.
- 规则 (3) 选择语句"switch (A) $\{$ case a_1:S_1 break; case a_2 : S_2 break; $\cdots$ break; case a_m : S_m; break;$\}$"
  - 需要 $\max(T_{s1}, T_{s2}, \cdots, T_{sm})$ 的时间;
  - 其中 T_{si} 是执行语句 S_i 所需要的时间, $i = l, 2, \cdots, m$.
- 规则 (4) 访问一个数组的单个分量或一个记录的单个域, 只需要 1 个单位时间.

基本运算、语句的运行时间分析

- 规则 (5) 执行一个 for 循环语句需要的时间等于执行该循环体所需要的时间乘上循环的次数.
- 规则 (6) 执行一个 while 循环语句"while (C) S;", 需要的时间等于计算条件表达式 C 需要的时间与执行循环 S 体需要的时间之和乘以循环的次数。与规则 5 不同，这里的循环次数是隐含的.



基本运算、语句的运行时间分析

- 规则 (7) 对于 goto 语句
  - 为了便于表达从循环体的中途跳转到循环体的结束或跳转到循环语句的后面语句, 可以假设它不需要任何额外的时间;
  - 如果程序滥用了 goto 语句, 即控制转移到前面的语句, 那么情况将变得复杂. 当这种转移造成某种循环时，只要与其他循环不交叉，保持循环的内外嵌套，则可按规则 (1)-(6) 进行分析;
  - 当由于使用 goto 语句而使程序结构混乱时，建议改写程序然后再分析.



基本运算、语句的运行时间分析

- 规则 (8) 对于函数调用语句，它们需要的时间包括两部分，一部分用于实现控制转移，另一部分用于执行函数本身.
  - 无递归调用: 可根据函数调用的层次，由里向外运用规则 (1)-(7) 逐层分析，直到计算出最外层的运行时间;
  - 函数出现直接或间接的递归调用，则由里向外逐层分析行不通.
    - 这时可以对其中各个递归过程 (函数)，所需要的时间假设为一个待定函数;
    - 然后根据过程 (函数) 间的关系建立待定函数之间的递归关系，得到递归方程;
    - 最后求解递归方程解的渐近阶，确定最坏情况下时间复杂性的渐近阶.



算法案例——查找

已知不重复且已经按从小到大排好的 m 个整数的数组 $A[0:m-1]$. 对于给定的整数 c, 要求寻找一个下标 i, 使得 $A[i]=c$; 若找不到, 则返回 -1. (为简单起见, 设 $m=2^k$, k 是一个确定的非负整数, 并假设没有重复元素)



算法案例——查找

已知不重复且已经按从小到大排好的 m 个整数的数组 $A[0:m-1]$. 对于给定的整数 c, 要求寻找一个下标 i, 使得 $A[i]=c$; 若找不到, 则返回 -1. (为简单起见, 设 $m=2^k$, k 是一个确定的非负整数, 并假设没有重复元素)

- 问题的规模以 m 计算;
- 算法用到的元运算有赋值 (a)、测试比较 (t)、加减法 (s)、除法 (d) 等, 它们每执行一次所需的时间常数分别为 a, t, s, d.



算法 A: 顺序查找

```
1  // input: A[0:m-1], increasing order
2  // output: the index of element c
3  int seq_search(int m, const int* A, int c)
4  {
5    int i=0;
6
7    while (i<m and A[i]<c) {
8      ++i;
9    }
10
11   if (i<m and A[i]==c) return i;
12
13   return -1;
14 }
```



算法 A: 顺序查找

```
1  // input: A[0:m-1], increasing order
2  // output: the index of element c
3  int seq_search(int m, const int* A, int c)
4  {
5    int i=0;
6
7    while (i<m and A[i]<c) {
8      ++i;
9    }
10
11   if (i<m and A[i]==c) return i;
12
13   return -1;
14 }
```

- 假设 $A[j] = c$
- 赋值次数: $1 + j$
- 比较次数: $2(j + 2)$
- 加法次数: j
- 总时间:

$$(j+1)a + 2(j+2)t + js$$
$$= j(a + 2t + s) + a + 4t$$



算法 A: 顺序查找

- **最坏时间**: $A[m-1]=c$,

$$m(a+2t+s)+2t-s \tag{1}$$

- **平均时间**: 假设每个位置出现 c 的概率均等, 即 $P(A[i]=c)=1/m$,

$$\begin{aligned}\sum_{j=0}^{m-1}\frac{j(a+2t+s)+a+4t}{m} &= \frac{a+2t+s}{m}\sum_{j=0}^{m-1}j+(a+4t)\\ &= \frac{a+2t+s}{2}m+(a/2+3t-s/2)\end{aligned} \tag{2}$$



算法 B: 二分查找 (折半查找)

```
1  // input: A[0:m-1], increasing order
2  // output: find the index of element c
3  int bisearch(int m, const int* A, int c)
4  {
5    int l(0), u(m-1), i;
6
7    while (l<=u) {
8      i = (l+u)/2;
9      if (A[i]==c) {
10       return i; // find
11     } else if (A[i]>c) {
12       u = i-1;
13     } else {
14       l = i+1;
15     }//if
16   }//while
17
18   return -1;
19 }
```



算法 B: 二分查找 (折半查找)

```
1  // input: A[0:m-1], increasing order
2  // output: find the index of element c
3  int bisearch(int m, const int* A, int c)
4  {
5    int l(0), u(m-1), i;
6
7    while (l<=u) {
8      i = (l+u)/2;
9      if (A[i]==c) {
10       return i; // find
11     } else if (A[i]>c) {
12       u = i-1;
13     } else {
14       l = i+1;
15     }//if
16   }//while
17
18   return -1;
19 }
```

- **循环次数**: $r \le \lceil \log_2 m \rceil + 1$
- **赋值次数**: $2 + 2(r-1) + 1$
- **比较次数**: $3(r-1) + 2$
- **加减法次数**: $2(r-1) + 1$
- **除法次数**: r
- **总时间**:

$$(2r+1)a + (3r-1)t + (2r-1)s + rd$$
$$= r(2a+3t+2s+d) + a - t - s$$



两种方法对比

- **算法 A 的最坏情况**: $T_A(m, max) := m(a + 2t + s) + 2t - s$.
- **算法 A 的平均情况**:

$$T_A(m, ave) := m(a + 2t + s)/2 + (a/2 + 3t - s/2).$$

- **算法 B 的最坏情况**:

$$T_B(m, max) := (\log_2 m)(2a + 3t + 2s + d) + (3a + 2t + s + d).$$

- **当 m 足够大时**: $m \gg \log_2 m$,

$$T_A(m, max) > T_A(m, ave) \gg T_B(m, max).$$



算法复杂性度量

- 如果分别用 N、I 和 A 来表示算法要解问题的规模、算法的输入和算法本身，用 C 表示算法的复杂性，则:

$$C = F(N, I, A).$$

  其中 $F(N, I, A)$ 是 N、I、A 的一个确定的三元函数.
- 将时间复杂性和空间复杂性分开表示:
  - 时间复杂性: $T = T(N, I, A)$;
  - 空间复杂性: $S = S(N, I, A)$.
- 通常将 A(算法本身) 隐含在复杂性函数当中，则上式简写为:

$$T = T(N, I),$$
$$S = S(N, I).$$



算法时间复杂性具体化

- $T(N, I)$ 为算法在一台抽象的计算机上运行所需的时间;
- 设此抽象的计算机所提供的元运算有 k 种，分别记为 $O_1, O_2, \cdots, O_k$;
- 再设这些元运算执行一次所需时间分别为 $t_1, t_2, \cdots, t_k$;
- 给定算法 A，设经统计用到元运算 O_i 的次数为 e_i, $i = 1, 2, .., k$;
- $\forall i,\ 1 \leq i \leq k$, e_i 是 N 和 I 的函数，即 $e_i = e_i(N, I)$;
- 则时间复杂度为:

$$T(N, I) = \sum_{i=1}^{k} t_i \cdot e_i(N, I)$$

其中 $t_i, i = 1, 2, \cdots, k$, 是与 N、I 无关的常数.

算法时间复杂性具体化

最坏情况: $T_{max}(N)$; 最好情况: $T_{min}(N)$; 平均情况: $T_{ave}(N)$.

$$T_{max}(N) := \max_{I \in D_N} T(N, I) = \max_{I \in D_N} \sum_{i=1}^{k} t_i e_i(N, I) = \sum_{i=1}^{k} t_i e_i(N, I^*) = T(N, I^*)$$

$$T_{min}(N) := \min_{I \in D_N} T(N, I) = \min_{I \in D_N} \sum_{i=1}^{k} t_i e_i(N, I) = \sum_{i=1}^{k} t_i e_i(N, \tilde{I}) = T(N, \tilde{I})$$

$$T_{ave}(N) := \sum_{I \in D_N} P(I) \cdot T(N, I) = \sum_{I \in D_N} P(I) \sum_{i=1}^{k} t_i e_i(N, I)$$

其中 D_N 是规模为 N 的合法输入的集合, I^* 是 D_N 中使 $T(N, I^*)$ 达到 $T_{max}(N)$ 的合法输入; $\tilde{I}$ 是 D_N 中使 $T(N, \tilde{I})$ 到 $T_{min}(N)$ 的合法输入, $P(I)$ 是 D_N 中输入 I 的发生概率.



复杂性渐近性态

- 设 $T(n)$ 是算法 A 的时间复杂性函数.
- 当 n 单调增加且趋于 ∞ 时, $T(n)$ 也将单调增加趋于 ∞.
- 对于 $T(n)$, 如果存在 $T'(n)$, 使得当 $N \to \infty$ 时有:

$$\frac{T(n) - T'(n)}{T(n)} \to 0.$$

- 那么, $T'(n)$ 是 $T(n)$ 当 $n \to \infty$ 时的渐近性态, 或叫 $T'(n)$ 为算法 A 当 $n \to \infty$ 的渐近复杂性.
- 直观上, $T'(n)$ 是 $T(n)$ 中略去低阶项所留下的主项.
- 比如当 $T(n) = 3n^2 + 4nlog_2n + 7$, $T'(n) = 3n^2$.
- $T'(n)$ 比 $T(n)$ 简洁直观.



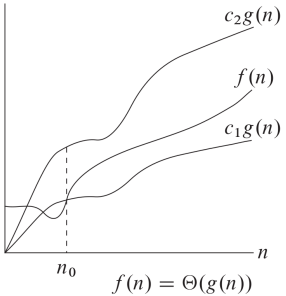
渐近符号

- 算法效率的主要指标是基本操作次数的增长次数
- 为了对这些增长次数进行比较和归类，计算机科学家们使用了 3 种符号:
  - O: 上界 (asymptotic upper bound)
  - Ω: 下界 (asymptotic lower bound)
  - Θ: 相等 (asymptotically tight bound)



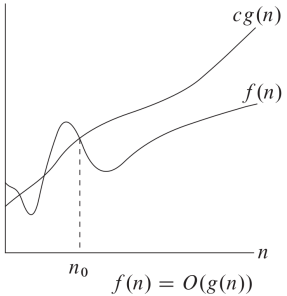
渐近符号

- $O(g(n)) = \{f(n) \mid \exists c, n_0, \forall n \geq n_0,\ f(n) \leq cg(n)\}$.
- $\Omega(g(n)) = \{f(n) \mid \exists c, n_0, \forall n \geq n_0,\ f(n) \geq cg(n)\}$.
- $\Theta(g(n)) = \{f(n) \mid \exists c_1, c_2, n_0, \forall n \geq n_0,\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\}$.



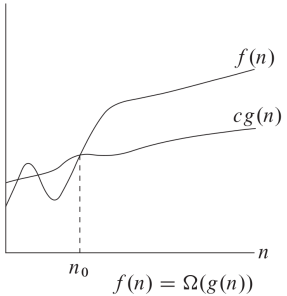
$f(n) = \Theta(g(n))$

(a)



$f(n) = O(g(n))$

(b)

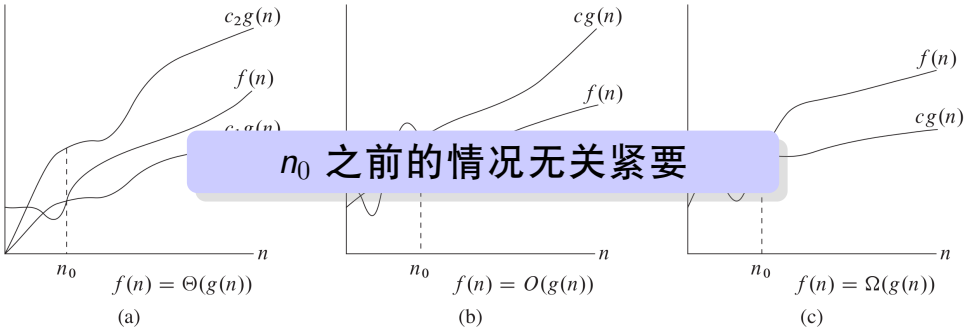


$f(n) = \Omega(g(n))$

(c)

渐近符号

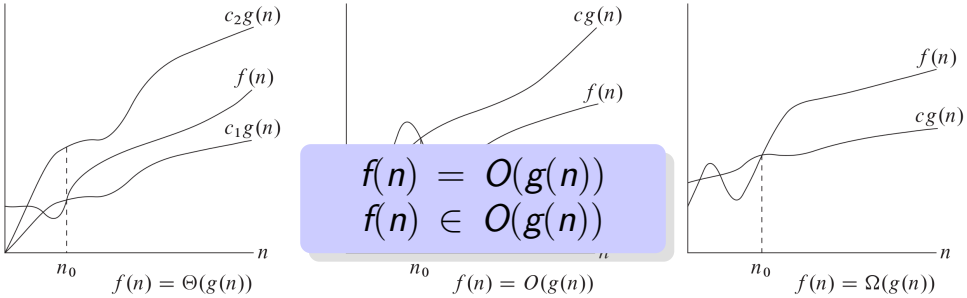
- $O(g(n)) = \{f(n) \mid \exists c, n_0, \forall n \geq n_0,\ f(n) \leq cg(n)\}$.
- $\Omega(g(n)) = \{f(n) \mid \exists c, n_0, \forall n \geq n_0,\ f(n) \geq cg(n)\}$.
- $\Theta(g(n)) = \{f(n) \mid \exists c_1, c_2, n_0, \forall n \geq n_0,\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\}$.



n_0 之前的情况无关紧要

渐近符号

- $O(g(n)) = \{f(n) \mid \exists c, n_0, \forall n \geq n_0,\ f(n) \leq cg(n)\}$.
- $\Omega(g(n)) = \{f(n) \mid \exists c, n_0, \forall n \geq n_0,\ f(n) \geq cg(n)\}$.
- $\Theta(g(n)) = \{f(n) \mid \exists c_1, c_2, n_0, \forall n \geq n_0,\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\}$.

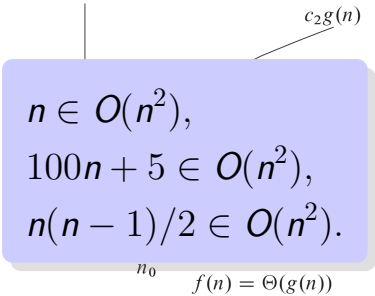


$$f(n) = O(g(n))$$
$$f(n) \in O(g(n))$$

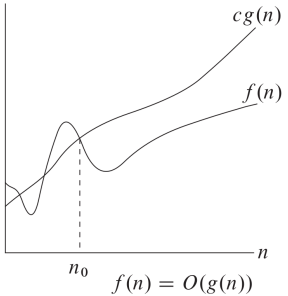
渐近符号

- $O(g(n)) = \{f(n) \mid \exists c, n_0, \forall n \geq n_0,\ f(n) \leq cg(n)\}$.
- $\Omega(g(n)) = \{f(n) \mid \exists c, n_0, \forall n \geq n_0,\ f(n) \geq cg(n)\}$.
- $\Theta(g(n)) = \{f(n) \mid \exists c_1, c_2, n_0, \forall n \geq n_0,\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\}$.

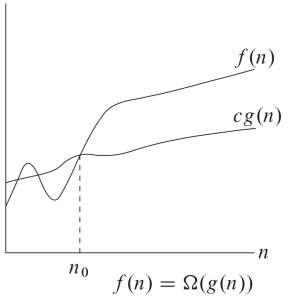
$n \in O(n^2)$,
$100n + 5 \in O(n^2)$,
$n(n-1)/2 \in O(n^2)$.



(a)



(b)

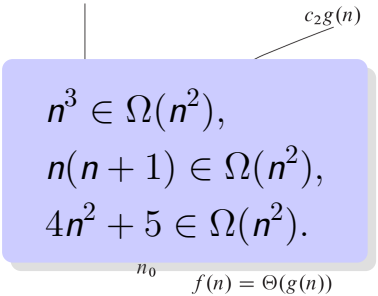


(c)

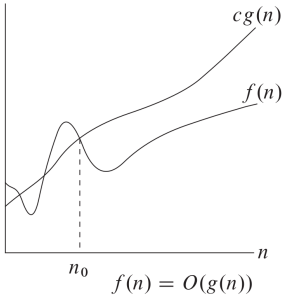
渐近符号

- $O(g(n)) = \{f(n) \mid \exists c, n_0, \forall n \geq n_0,\ f(n) \leq cg(n)\}$.
- $\Omega(g(n)) = \{f(n) \mid \exists c, n_0, \forall n \geq n_0,\ f(n) \geq cg(n)\}$.
- $\Theta(g(n)) = \{f(n) \mid \exists c_1, c_2, n_0, \forall n \geq n_0,\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\}$.

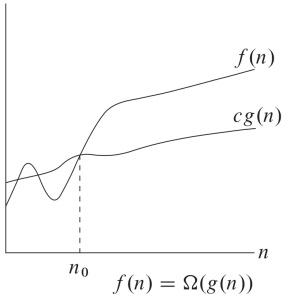
$n^3 \in \Omega(n^2)$,
$n(n+1) \in \Omega(n^2)$,
$4n^2 + 5 \in \Omega(n^2)$.



(a)



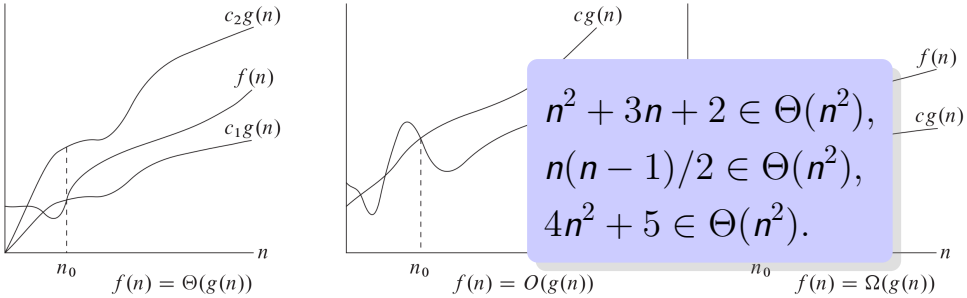
(b)



(c)

渐近符号

- $O(g(n)) = \{f(n) \mid \exists c, n_0, \forall n \geq n_0,\ f(n) \leq cg(n)\}$.
- $\Omega(g(n)) = \{f(n) \mid \exists c, n_0, \forall n \geq n_0,\ f(n) \geq cg(n)\}$.
- $\Theta(g(n)) = \{f(n) \mid \exists c_1, c_2, n_0, \forall n \geq n_0,\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\}$.



渐近符号

- 当 $n \geq 1$, $3n \leq 4n$, 则 $3n = O(n)$;
- 当 $n \geq 1$, $n + 1024 \leq 1025n$, 则 $n + 1024 = O(n)$;
- 当 $n \geq 10$, $2n^2 + 11n - 10 \leq 3n^2$, 则 $2n^2 + 11n - 10 = O(n^2)$;
- 当 $n \geq 1$, $n^2 \leq n^3$, 则 $n^2 = O(n^3)$;



渐近符号

- 当 $n \geq 1$, $3n \leq 4n$, 则 $3n = O(n)$;
- 当 $n \geq 1$, $n + 1024 \leq 1025n$, 则 $n + 1024 = O(n)$;
- 当 $n \geq 10$, $2n^2 + 11n - 10 \leq 3n^2$, 则 $2n^2 + 11n - 10 = O(n^2)$;
- 当 $n \geq 1$, $n^2 \leq n^3$, 则 $n^2 = O(n^3)$;
- 反例: $n^3 \neq O(n^2)$.
  - 若不然, 则存在常数 $c > 0$ 和自然数 n_0, 使得当 $n \geq n_0$ 时有 $n^3 \leq cn^2$, 即 $n \leq c$, 显然当 $n = \max(n_0, \lfloor c \rfloor + 1)$ 时这个不等式不成立, 所以 $n^3 \neq O(n^2)$.



渐近符号的运算规则

- $O(f) + O(g) = O(\max(f, g))$;
- $O(f) + O(g) = O(f + g)$;
- $O(f) \cdot O(g) = O(f \cdot g)$;
- 如果 $g = O(f)$, 则 $O(f) + O(g) = O(f)$;
- $O(C \cdot f) = O(f)$, 其中 C 是一个正的常数;
- $f = O(f)$, 是自身的上界.



其他渐近符号

- $o(f)$
- $\omega(f)$
- $\theta(f)$



举例: 分析二重循环的渐近复杂性

```
1 for (int i=1; i<=n; ++i) {
2   for (int j=1; j<=i; ++j) {
3     S1; S2; S2; S4; //Si是单一赋值语句
4   }
5 }
```



举例: 分析二重循环的渐近复杂性

```
1 for (int i=1; i<=n; ++i) {
2   for (int j=1; j<=i; ++j) {
3     S1; S2; S2; S4; //Si是单一赋值语句
4   }
5 }
```

- 内循环: 只需要 $O(1)$, 则内循环只需要:

$$\sum_{j=1}^{i} O(1) = O(1)i = O(i).$$

- 总的时间复杂性:

$$\sum_{i=1}^{n} O(i) = O(\sum_{i}^{n} i) = O(\frac{n(n+1)}{2}) = O(n^2).$$

算法与渐近时间复杂性的关系

让六种算法在 C_1 和 C_2 两台计算机上运行，并设 C_2 的计算速度是 C_1 的 10 倍，比较相同时间下求解规模之间的关系.

算法	$T'(n)$	C_1 上可解的规模 n_1	C_2 上可解的规模 n_2	n_1 与 n_2 的关系
A_1	n	n_{11}	n_{21}	$n_{21} = 10n_{11}$
A_2	$n\log n$	n_{12}	n_{22}	$n_{22} \approx 10n_{12}$
A_3	n^2	n_{13}	n_{23}	$n_{23} = \sqrt{10}n_{13}$
A_4	n^3	n_{14}	n_{24}	$n_{24} = \sqrt[3]{10}n_{14}$
A_5	2^n	n_{15}	n_{25}	$n_{25} = n_{15} + \log_2 10$
A_6	$n!$	n_{16}	n_{26}	$n_{26} = n_{16} + \varepsilon$



算法与渐近时间复杂性的关系

让六种算法在 C_1 和 C_2 两台计算机上运行，并设 C_2 的计算速度是 C_1 的 10 倍，比较相同时间下求解规模之间的关系.

算法	$T'(n)$	C_1 上可解的规模 n_1	C_2 上可解的规模 n_2	n_1 与 n_2 的关系
A_1	n	n_{11}	n_{21}	$n_{21} = 10n_{11}$
A_2	$n\log n$	n_{12}	n_{22}	$n_{22} \approx 10n_{12}$
A_3	n^2	n_{13}	n_{23}	$n_{23} = \sqrt{10}n_{13}$
A_4	n^3	n_{14}	n_{24}	$n_{24} = \sqrt[3]{10}n_{14}$
A_5	2^n	n_{15}	n_{25}	$n_{25} = n_{15} + \log_2 10$
A_6	$n!$	n_{16}	n_{26}	$n_{26} = n_{16} + \varepsilon$

- 高效的算法 A_1: 计算机性能增长 10 倍，可求解的规模同步增长 10 倍;
- 低效的算法 A_3: 计算机性能增长 10 倍，只换取问题规模增加 $\log_2 10$.



算法与渐近时间复杂性的关系

让六种算法在 C_1 和 C_2 两台计算机上运行，并设 C_2 的计算速度是 C_1 的 10 倍，比较相同时间下求解规模之间的关系.

算法	$T'(n)$	C_1 上可解的规模 n_1	C_2 上可解的规模 n_2	n_1 与 n_2 的关系
A_1	n	n_{11}	n_{21}	$n_{21} = 10n_{11}$
A_2	$n\log n$	n_{12}	n_{22}	$n_{22} \approx 10n_{12}$
A_3	n^2	n_{13}	n_{23}	$n_{23} = \sqrt{10}n_{13}$
A_4	n^3	n_{14}	n_{24}	$n_{24} = \sqrt[3]{10}n_{14}$
A_5	2^n	n_{15}	n_{25}	$n_{25} = n_{15} + \log_2 10$
A_6	$n!$	n_{16}	n_{26}	$n_{26} = n_{16} + \varepsilon$

设计高效算法很重要

- 高效的算法 A_1：计算机性能增长 10 倍，可求解的规模同步增长 10 倍；
- 低效的算法 A_3：计算机性能增长 10 倍，只换取问题规模增加 $\log_2 10$.



算法与渐近时间复杂性的关系

讨论算法复杂性渐近阶的同时，须注意两点：



算法与渐近时间复杂性的关系

讨论算法复杂性渐近阶的同时，须注意两点：

- 复杂性的渐近阶较低的算法比渐近阶较高的算法更有效



算法与渐近时间复杂性的关系

讨论算法复杂性渐近阶的同时，须注意两点：

- 复杂性的渐近阶较低的算法比渐近阶较高的算法更有效
  - 这个结论，只是在问题的求解规模充分大时才成立.
  - 比如算法 A_4 比 A_5 有效只是在 $n^3 < 2^n$ 时才成立.
  - 设 c 是方程 $n^3 = 2^n$ 的解，当 $n < c$ 时，A_5 反而比 A_4 有效.
  - 故对于规模小的问题，不能盲目地选用复杂性阶比较低的算法.
  - 复杂性阶比较低的算法在规模小时不一定比复杂性阶比较高的算法更有效.
  - 另外，规模小时，决定工作效率的可能不是算法的效率而是算法的简单性，哪一种算法简单、易实现，就选用哪一种算法.



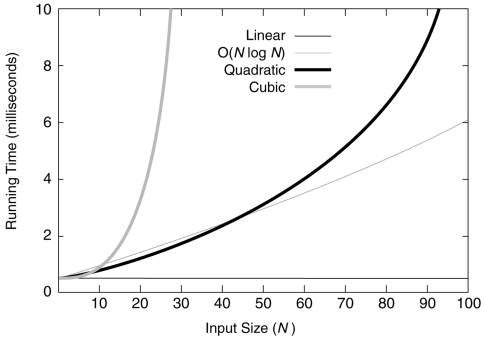
算法与渐近时间复杂性的关系

讨论算法复杂性渐近阶的同时，须注意两点：

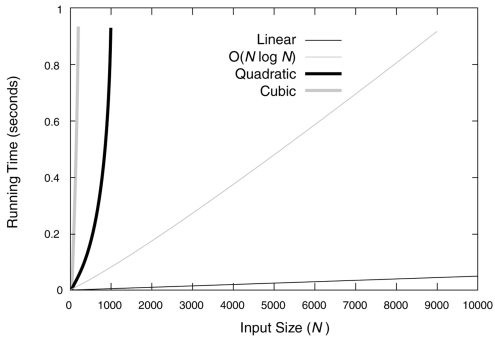
- 复杂性的渐近阶较低的算法比渐近阶较高的算法更有效
  - 这个结论，只是在问题的求解规模充分大时才成立.
  - 比如算法 A_4 比 A_5 有效只是在 $n^3 < 2^n$ 时才成立.
  - 设 c 是方程 $n^3 = 2^n$ 的解，当 $n < c$ 时，A_5 反而比 A_4 有效.
  - 故对于规模小的问题，不能盲目地选用复杂性阶比较低的算法.
  - 复杂性阶比较低的算法在规模小时不一定比复杂性阶比较高的算法更有效.
  - 另外，规模小时，决定工作效率的可能不是算法的效率而是算法的简单性，哪一种算法简单、易实现，就选用哪一种算法.
- 当两个算法的渐近复杂性的阶相同时，必须进一步考察表达式中常数因子才能判别优劣. 常数因子小的优于常数因子大的算法. 比如 $n\log n/100$ 的算法显然比 $100n\log n$ 的算法更有效.

算法与渐近时间复杂性的关系

小规模



中等规模



空间复杂性

- 可参照时间复杂性的分析;
- 出现函数递归调用时要考虑到其中隐含的存储空间的额外开销.



递归算法的时间复杂性

```
1 // 输入：数组 A 存储 n 个整数，n>0
2 // 输出：A 中的最大值
3 int recursive_max(int n, int* A)
4 {
5   if (n==1) return A[0];
6   return max(recursive_max(n-1, A), A[n-1]);
7 }
```



递归算法的时间复杂性

```
1 // 输入：数组 A 存储 n 个整数，n>0
2 // 输出：A 中的最大值
3 int recursive_max(int n, int* A)
4 {
5   if (n==1) return A[0];
6   return max(recursive_max(n-1, A), A[n-1]);
7 }
```

$$T(n) = \begin{cases} c_1 & n = 1 \\ c_2 + T(n-1) & n > 1 \end{cases}$$



递归算法的时间复杂性

```
1 // 输入: 数组 A 存储 n 个整数, n>0
2 // 输出: A 中的最大值
3 int recursive_max(int n, int* A)
4 {
5   if (n==1) return A[0];
6   return max(recursive_max(n-1, A), A[n-1]);
7 }
```

$$T(n) = \begin{cases} c_1 & n = 1 \\ c_2 + T(n-1) & n > 1 \end{cases}$$

这是一个关于 $T(n)$ 的递归方程, 利用迭代法, 易得:

$$T(n) = c_2 n + c_2 + c_1 = \Theta(n).$$

二分查找的时间复杂性分析

```
1 // here l>=0
2 int recursive_bisearch(int l, int u, int* A, int c)
3 {
4   if (l>u) return -1; //not found
5   int m = (l+u)/2;
6   if (A[m]==c) {return m;}
7   else if (A[m]<c) {return recursive_bisearch(m+1, u, A, c);}
8   else {return recursive_bisearch(l, m-1, A, c);}
9 }
```



二分查找的时间复杂性分析

```
1 // here l>=0
2 int recursive_bisearch(int l, int u, int* A, int c)
3 {
4   if (l>u) return -1; //not found
5   int m = (l+u)/2;
6   if (A[m]==c) {return m;}
7   else if (A[m]<c) {return recursive_bisearch(m+1, u, A, c);}
8   else {return recursive_bisearch(l, m-1, A, c);}
9 }
```

$$T(m) = \begin{cases} c_1 & m = 0 \\ c_2 + T(0) & m = 1 \\ c_3 + T(m/2) & m > 1 \end{cases}$$



二分查找的时间复杂性分析

```
1 // here l>=0
2 int recursive_bisearch(int l, int u, int* A, int c)
3 {
4   if (l>u) return -1; //not found
5   int m = (l+u)/2;
6   if (A[m]==c) {return m;}
7   else if (A[m]<c) {return recursive_bisearch(m+1, u, A, c);}
8   else {return recursive_bisearch(l, m-1, A, c);}
9 }
```

$$T(m) = \begin{cases} c_1 & m = 0 \\ c_2 + T(0) & m = 1 \\ c_3 + T(m/2) & m > 1 \end{cases}$$

这是一个关于 $T(m)$ 的递归方程, 利用迭代法, 易得:

$$T(m) = c_3 \log m + c_2 + c_1 = \Theta(\log m).$$

递归方程的求解

- 代入法
- 迭代法（递归树法）
- 主方法（公式法）
- 差分方程法（求解特征方程）*
- 生成函数法（母函数法）*



代入法

代入法求解递归式分为两步:

- 猜测解的形式;
- 用数学归纳法求出解中的常数，并证明解是正确的.



代入法

$T(n) = 2T(\lfloor n/2 \rfloor) + n$. **猜测** $T(n)$ **上界为** $n \log n$.



代入法

$T(n) = 2T(\lfloor n/2 \rfloor) + n$. **猜测** $T(n)$ **上界为** $n \log n$.

利用数学归纳法证明 (Inductive Step)

$$T(\lfloor n/2 \rfloor) \leq c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor$$
$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$



代入法

$T(n) = 2T(\lfloor n/2 \rfloor) + n$. **猜测** $T(n)$ **上界为** $n \log n$.

利用数学归纳法证明 (Inductive Step)

$$
\begin{aligned}
T(\lfloor n/2 \rfloor) &\le c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor \\
T(n) &= 2T(\lfloor n/2 \rfloor) + n \\
&\le 2c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n
\end{aligned}
$$



代入法

$T(n) = 2T(\lfloor n/2 \rfloor) + n$. **猜测** $T(n)$ **上界为** $n \log n$.

利用数学归纳法证明 (Inductive Step)

$$
\begin{aligned}
T(\lfloor n/2 \rfloor) &\le c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor \\
T(n) &= 2T(\lfloor n/2 \rfloor) + n \\
&\le 2c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n \\
&\le cn \log n/2 + n
\end{aligned}
$$



代入法

$T(n) = 2T(\lfloor n/2 \rfloor) + n$. **猜测** $T(n)$ **上界为** $n \log n$.

利用数学归纳法证明 (Inductive Step)

$$
\begin{aligned}
T(\lfloor n/2 \rfloor) &\le c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor \\
T(n) &= 2T(\lfloor n/2 \rfloor) + n \\
&\le 2c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n \\
&\le cn \log n/2 + n
\end{aligned}
$$



代入法

$T(n) = 2T(\lfloor n/2 \rfloor) + n$. **猜测** $T(n)$ **上界为** $n \log n$.

利用数学归纳法证明 (Inductive Step)

$$
\begin{aligned}
T(\lfloor n/2 \rfloor) &\le c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor \\
T(n) &= 2T(\lfloor n/2 \rfloor) + n \\
&\le 2c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n \\
&\le cn \log n/2 + n \\
&= cn \log n + n(1 - c)
\end{aligned}
$$



代入法

$T(n) = 2T(\lfloor n/2 \rfloor) + n$. **猜测** $T(n)$ **上界为** $n \log n$.

利用数学归纳法证明 (Inductive Step)

$$
\begin{aligned}
T(\lfloor n/2 \rfloor) &\le c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor \\
T(n) &= 2T(\lfloor n/2 \rfloor) + n \\
&\le 2c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n \\
&\le cn \log n/2 + n \\
&= cn \log n + n(1 - c) \\
&\le cn \log n \qquad (\text{when } c \ge 1)
\end{aligned}
$$



代入法

$T(n) = 2T(\lfloor n/2 \rfloor) + n$. **猜测** $T(n)$ **上界为** $n \log n$.

利用数学归纳法证明 (Inductive Step)

$$
\begin{aligned}
T(\lfloor n/2 \rfloor) &\le c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor \\
T(n) &= 2T(\lfloor n/2 \rfloor) + n \\
&\le 2c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n \\
&\le cn \log n/2 + n \\
&= cn \log n + n(1-c) \\
&\le cn \log n \qquad (\text{when } c \ge 1)
\end{aligned}
$$

Basic Step: if $T(1) = c_1$, $T(2) = 4 \le (2 \log 2)c$, when $c \ge 2$.



代入法

一个递归方程类似于已知解的方程，推测它有类似的解是合理的.



代入法

一个递归方程类似于已知解的方程，推测它有类似的解是合理的.

$$T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$$

猜测 $T(n)$ 上界为 $n \log n$.



代入法

一个递归方程类似于已知解的方程，推测它有类似的解是合理的.

$$T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$$

猜测 $T(n)$ 上界为 $n \log n$.

当 n 充分大时，$T(\lfloor n/2 \rfloor + 17) \approx T(\lfloor n/2 \rfloor)$.



代入法

从较宽松的界开始推测，逐步逼近精确界.

$T(n) = 2T(\lfloor n/2 \rfloor) + n$

估计其解的渐近下界，明显有

$$T(n) \geq n.$$

可推测 $T(n) = \Omega(n)$，并猜测 $T(n) = O(n^2)$，发现太松后，提高下界，降低上界，可得

$$T(n) = \Theta(n \log n).$$



代入法

作变元的替换有时会使一个未知其解的递归方程变成熟悉的方程，从而作逆变换可得到原方程的解.

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \log n.$$

- 定义 $m = \log n$.
- $T(2^m) = 2T(\lfloor 2^{m/2} \rfloor) + m$.
- 定义 $S(m) = T(2^m)$.
- $S(m) = 2S(m/2) + m$. (把 m 限制在正偶数集上)
- $T(n) = T(2^m) = S(m) = m \log m = O(\log n \log \log n)$.



迭代法 (递归树法)

- 通过反复迭代，将递归方程的右端变换成一个级数;
- 然后求级数的和，再估计和的渐近阶;
- 或不求级数的和而直接估计级数的渐近阶;
- 从而得到对递归方程解的渐近阶的估计.



迭代法 (递归树法)

$$T(n) = 3T(\lfloor n/4 \rfloor) + n.$$



迭代法 (递归树法)

$$T(n) = 3T(\lfloor n/4 \rfloor) + n.$$

$$\begin{aligned} T(n) &= n + 3(3T(\lfloor \lfloor n/4 \rfloor /4 \rfloor) + \lfloor n/4 \rfloor) \\ &= n + 3\lfloor n/4 \rfloor + 3^2(3T(\lfloor \lfloor \lfloor n/4 \rfloor /4 \rfloor /4 \rfloor) + \lfloor \lfloor n/4 \rfloor /4 \rfloor) \end{aligned}$$

迭代法 (递归树法)

$$T(n) = 3T(\lfloor n/4 \rfloor) + n.$$

$$\begin{aligned} T(n) &= n + 3(3T(\lfloor \lfloor n/4 \rfloor /4 \rfloor) + \lfloor n/4 \rfloor) \\ &= n + 3\lfloor n/4 \rfloor + 3^2(3T(\lfloor \lfloor \lfloor n/4 \rfloor /4 \rfloor /4 \rfloor) + \lfloor \lfloor n/4 \rfloor /4 \rfloor) \\ &= n + 3\lfloor n/4 \rfloor + 3^2\lfloor n/4^2 \rfloor + 27T(\lfloor n/4^3 \rfloor) \end{aligned}$$

迭代法 (递归树法)

$$T(n) = 3T(\lfloor n/4 \rfloor) + n.$$

$$\begin{aligned}
T(n) &= n + 3(3T(\lfloor \lfloor n/4 \rfloor /4 \rfloor) + \lfloor n/4 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2(3T(\lfloor \lfloor \lfloor n/4 \rfloor /4 \rfloor /4 \rfloor) + \lfloor \lfloor n/4 \rfloor /4 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2\lfloor n/4^2 \rfloor + 27T(\lfloor n/4^3 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2\lfloor n/4^2 \rfloor + \cdots + 3^{k-1}\lfloor n/4^{k-1} \rfloor + 3^k T(\lfloor n/4^k \rfloor)
\end{aligned}$$

迭代法 (递归树法)

$$T(n) = 3T(\lfloor n/4 \rfloor) + n.$$

$$\begin{aligned}
T(n) &= n + 3(3T(\lfloor \lfloor n/4 \rfloor /4 \rfloor) + \lfloor n/4 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2(3T(\lfloor \lfloor \lfloor n/4 \rfloor /4 \rfloor /4 \rfloor) + \lfloor \lfloor n/4 \rfloor /4 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2\lfloor n/4^2 \rfloor + 27T(\lfloor n/4^3 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2\lfloor n/4^2 \rfloor + \cdots + 3^{k-1}\lfloor n/4^{k-1} \rfloor + 3^k T(\lfloor n/4^k \rfloor) \\
&\leq n + 3/4n + (3/4)^2 n + \cdots + (3/4)^{k-1} n + 3^k T(1)
\end{aligned}$$

迭代法 (递归树法)

$$T(n) = 3T(\lfloor n/4 \rfloor) + n.$$

$$\begin{aligned}
T(n) &= n + 3(3T(\lfloor \lfloor n/4 \rfloor /4 \rfloor) + \lfloor n/4 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2(3T(\lfloor \lfloor \lfloor n/4 \rfloor /4 \rfloor /4 \rfloor) + \lfloor \lfloor n/4 \rfloor /4 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2\lfloor n/4^2 \rfloor + 27T(\lfloor n/4^3 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2\lfloor n/4^2 \rfloor + \cdots + 3^{k-1}\lfloor n/4^{k-1} \rfloor + 3^k T(\lfloor n/4^k \rfloor) \\
&\leq n + 3/4n + (3/4)^2 n + \cdots + (3/4)^{k-1} n + 3^k T(1) \\
&< 4n + 3^k T(1)
\end{aligned}$$

迭代法 (递归树法)

$$T(n) = 3T(\lfloor n/4 \rfloor) + n.$$

$$\begin{aligned}
T(n) &= n + 3(3T(\lfloor \lfloor n/4 \rfloor /4 \rfloor) + \lfloor n/4 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2(3T(\lfloor \lfloor \lfloor n/4 \rfloor /4 \rfloor /4 \rfloor) + \lfloor \lfloor n/4 \rfloor /4 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2\lfloor n/4^2 \rfloor + 27T(\lfloor n/4^3 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2\lfloor n/4^2 \rfloor + \cdots + 3^{k-1}\lfloor n/4^{k-1} \rfloor + 3^k T(\lfloor n/4^k \rfloor) \\
&\leq n + 3/4n + (3/4)^2 n + \cdots + (3/4)^{k-1} n + 3^k T(1) \\
&< 4n + 3^k T(1)
\end{aligned}$$

其中 $\lfloor n/4^k \rfloor = 1$, **即** $k = \lfloor \log_4 n \rfloor$, $\lfloor x \rfloor \leq x$.

迭代法 (递归树法)

$$T(n) = 3T(\lfloor n/4 \rfloor) + n.$$

$$\begin{aligned}
T(n) &= n + 3(3T(\lfloor \lfloor n/4 \rfloor /4 \rfloor) + \lfloor n/4 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2(3T(\lfloor \lfloor \lfloor n/4 \rfloor /4 \rfloor /4 \rfloor) + \lfloor \lfloor n/4 \rfloor /4 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2\lfloor n/4^2 \rfloor + 27T(\lfloor n/4^3 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2\lfloor n/4^2 \rfloor + \cdots + 3^{k-1}\lfloor n/4^{k-1} \rfloor + 3^k T(\lfloor n/4^k \rfloor) \\
&\le n + 3/4n + (3/4)^2 n + \cdots + (3/4)^{k-1} n + 3^k T(1) \\
&< 4n + 3^k T(1) \le 4n + 3^{\log_4 n} T(1)
\end{aligned}$$

其中 $\lfloor n/4^k \rfloor = 1$, **即** $k = \lfloor \log_4 n \rfloor$, $\lfloor x \rfloor \le x$.

迭代法 (递归树法)

$$T(n) = 3T(\lfloor n/4 \rfloor) + n.$$

$$\begin{aligned}
T(n) &= n + 3(3T(\lfloor \lfloor n/4 \rfloor /4 \rfloor) + \lfloor n/4 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2(3T(\lfloor \lfloor \lfloor n/4 \rfloor /4 \rfloor /4 \rfloor) + \lfloor \lfloor n/4 \rfloor /4 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2\lfloor n/4^2 \rfloor + 27T(\lfloor n/4^3 \rfloor) \\
&= n + 3\lfloor n/4 \rfloor + 3^2\lfloor n/4^2 \rfloor + \cdots + 3^{k-1}\lfloor n/4^{k-1} \rfloor + 3^k T(\lfloor n/4^k \rfloor) \\
&\le n + 3/4n + (3/4)^2 n + \cdots + (3/4)^{k-1} n + 3^k T(1) \\
&< 4n + 3^k T(1) \le 4n + 3^{\log_4 n} T(1) = 4n + 3^{\log_4 n} T(1)
\end{aligned}$$

其中 $\lfloor n/4^k \rfloor = 1$, **即** $k = \lfloor \log_4 n \rfloor$, $\lfloor x \rfloor \le x$.

迭代法 (递归树法)

$$T(n) = 3T(\lfloor n/4 \rfloor) + n.$$

$$\begin{aligned}
T(n) &= n + 3(3T(\lfloor \lfloor n/4 \rfloor /4 \rfloor) + \lfloor n/4 \rfloor) \\
&= \\
&= \\
&= \qquad\qquad \lfloor n/4^k \rfloor) \\
&\le n + 3/4n + (3/4)^2 n + \cdots + (3/4)^{k-1} n + 3^k T(1) \\
&< 4n + 3^k T(1) \le 4n + 3^{\log_4 n} T(1) = 4n + 3^{\log_4 n} T(1)
\end{aligned}$$

$$T(n) < 4n + 3^{\log_4 3 \log_3 n} T(1) = 4n + n^{\log_4 3} T(1)$$

$$T(n) = O(n)$$

其中 $\lfloor n/4^k \rfloor = 1$, **即** $k = \lfloor \log_4 n \rfloor$, $\lfloor x \rfloor \le x$.

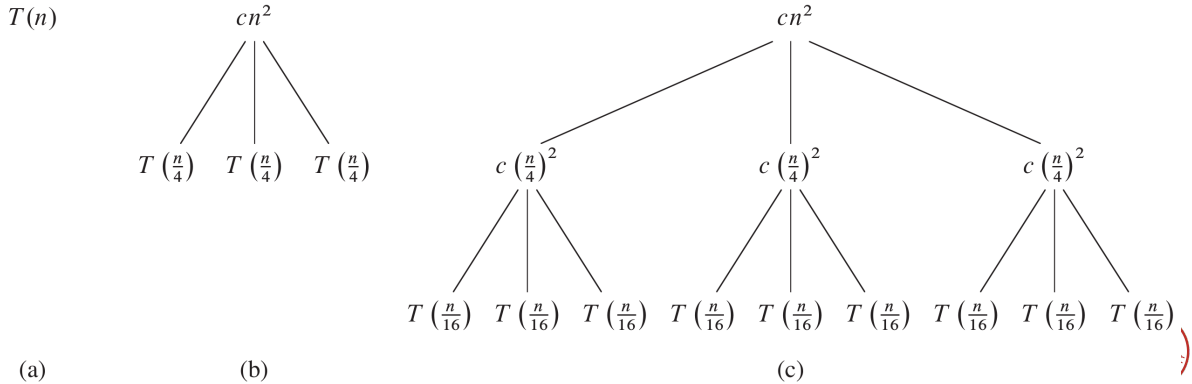
迭代法 (递归树法)

$$T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2).$$



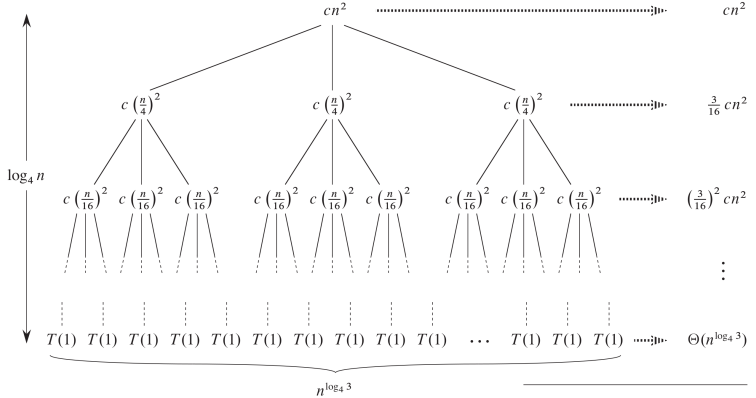
迭代法 (递归树法)

$$T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2).$$



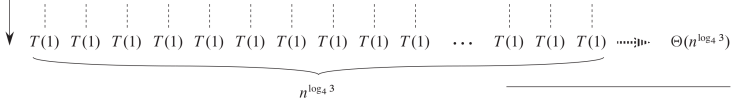
迭代法 (递归树法)

$$T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2).$$



迭代法 (递归树法)

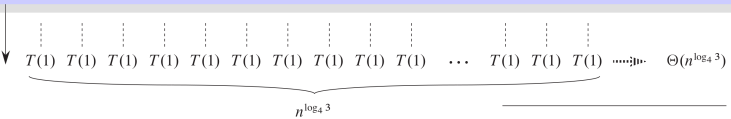
$$T(n) = cn^2 + \frac{3}{16}cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n-1} cn^2 + \Theta(n^{\log_4 3})$$



迭代法 (递归树法)

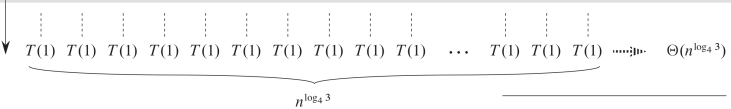
$$T(n) = cn^2 + \frac{3}{16}cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3})$$

$$= \frac{(3/16)^{\log_4 n} - 1}{3/16 - 1} cn^2 + \Theta(n^{\log_4 3})$$



迭代法 (递归树法)

$$
\begin{aligned}
T(n) &= cn^2 + \frac{3}{16}cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3}) \\
&= \frac{(3/16)^{\log_4 n} - 1}{3/16 - 1} cn^2 + \Theta(n^{\log_4 3}) \\
&< \frac{1}{1 - 3/16} cn^2 + \Theta(n^{\log_4 3}) = \frac{16}{13} cn^2 + \Theta(n^{\log_4 3})
\end{aligned}
$$



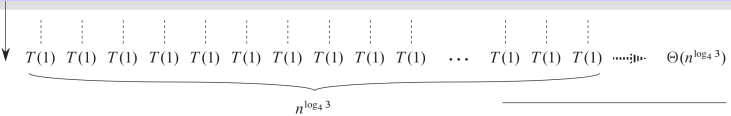
迭代法 (递归树法)

$$T(n) = cn^2 + \frac{3}{16}cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n-1} cn^2 + \Theta(n^{\log_4 3})$$

$$= \frac{(3/16}{3/16-1}$$

$$< \frac{1}{1-3/16}cn^2 + \Theta(n^{\log_4 3}) = \frac{16}{13}cn^2 + \Theta(n^{\log_4 3})$$

$$T(n) = O(n^2)$$



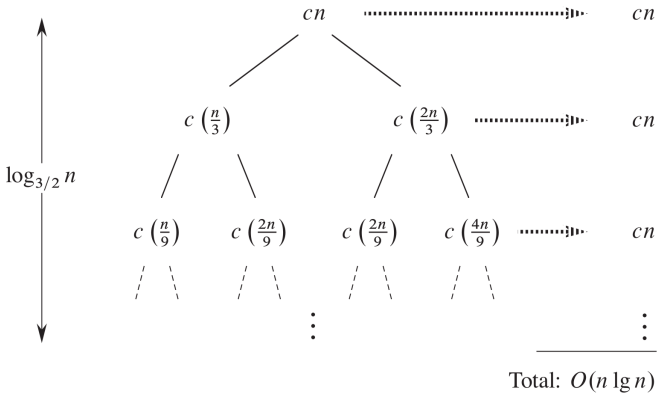
迭代法 (递归树法)

$$T(n) = T(n/3) + T(2n/3) + cn.$$



迭代法 (递归树法)

$$T(n) = T(n/3) + T(2n/3) + cn.$$



迭代法 (递归树法)

We assume $T(n) = O(n \log n)$.



迭代法 (递归树法)

We assume $T(n) = O(n \log n)$.

$$T(n) \leq T(n/3) + T(2/3n) + cn$$



迭代法 (递归树法)

We assume $T(n) = O(n \log n)$.

$$
\begin{aligned}
T(n) &\le T(n/3) + T(2/3n) + cn \\
&\le d(n/3)\log(n/3) + d(2n/3)\log(2n/3) + cn
\end{aligned}
$$



迭代法 (递归树法)

We assume $T(n) = O(n \log n)$.

$$
\begin{aligned}
T(n) &\le T(n/3) + T(2/3n) + cn \\
&\le d(n/3)\log(n/3) + d(2n/3)\log(2n/3) + cn \\
&= d(n/3)(\log n - \log 3) + d(2n/3)(\log n + \log 2/3) + cn
\end{aligned}
$$



迭代法 (递归树法)

We assume $T(n) = O(n \log n)$.

$$
\begin{aligned}
T(n) &\le T(n/3) + T(2/3n) + cn \\
&\le d(n/3)\log(n/3) + d(2n/3)\log(2n/3) + cn \\
&= d(n/3)(\log n - \log 3) + d(2n/3)(\log n + \log 2/3) + cn \\
&= d(n\log n) + dn(2/3 - \log 3) + cn
\end{aligned}
$$



迭代法 (递归树法)

We assume $T(n) = O(n \log n)$.

$$
\begin{aligned}
T(n) &\le T(n/3) + T(2/3n) + cn \\
&\le d(n/3)\log(n/3) + d(2n/3)\log(2n/3) + cn \\
&= d(n/3)(\log n - \log 3) + d(2n/3)(\log n + \log 2/3) + cn \\
&= d(n\log n) + dn(2/3 - \log 3) + cn \\
&\le dn \log n \qquad (\text{when } d \ge c/(\log 3 - 2/3))
\end{aligned}
$$



主方法 (公式法)

主方法为如下形式的递归式提供了一种“菜谱”式的求解方法:

$$T(n) = aT(n/b) + f(n)$$

其中 $a \geq 1, b \geq 1$ 是常数，$f(n)$ 是渐近正函数.



主方法 (公式法)

主方法为如下形式的递归式提供了一种“菜谱”式的求解方法:

$$T(n) = aT(n/b) + f(n)$$

其中 $a \geq 1, b \geq 1$ 是常数，$f(n)$ 是渐近正函数.

上式是一类分治法的时间复杂性所满足的递归关系，即一个规模为 n 的问题被分成规模均为 n/b 的 a 个子问题, 递归地求解这 a 个子问题，然后综合这 a 个子问题的解, 得到原问题的解.



主方法 (公式法)

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$$



主方法 (公式法)

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$$

- 若对于某常数 $\varepsilon > 0$, 有 $f(n) = O(n^{\log_b a - \varepsilon})$, 则 $T(n) = \Theta(n^{\log_b a})$.
- 若 $f(n) = \Theta(n^{\log_b a})$, 则 $T(n) = \Theta(n^{\log_b a} \cdot \log n)$.
- 若对于某常数 $\varepsilon > 0$, 有 $f(n) = \Omega(n^{\log_b a + \varepsilon})$ 且对于 $c < 1$ 和所有充分大的正整数 n 有 $af(n/b) \leq cf(n)$, 则 $T(n) = \Theta(f(n))$.



主方法 (公式法)

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$$

- 若
- 若
- 若

$\log_b a)$.

- 第一类情况，函数 $n^{\log_b a}$ 较大，则 $T(n) = \Theta(n^{\log_b a})$.
- 第二类情况，两个函数同阶，则 $T(n) = \Theta(n^{\log_b a} \log n)$.
- 第三类情况，函数 $f(n)$ 较大，则 $T(n) = \Theta(f(n))$.

所有充分大的正整数 n 有 $af(n/b) \le cf(n)$，则 $T(n) = \Theta(f(n))$.



主方法 (公式法)

$T(n) = 9T(n/3) + n$

- $a = 9, b = 3, f(n) = n.$
- $n^{\log_b a} = n^2.$
- 套用第一种情况, 得到 $T(n) = \Theta(n^2).$



主方法 (公式法)

$T(n) = T(2n/3) + 1$

- $a = 1, b = 2/3, f(n) = 1.$
- $n^{\log_b a} = n^0 = 1.$
- **套用第二种情况, 得到** $T(n) = \Theta(\log n).$



主方法 (公式法)

$T(n) = 3T(n/4) + n \log n$

- $a = 3, b = 4, f(n) = n \log n$.
- $n^{\log_b a} \approx n^{0.793}$.
- **套用第三种情况, 得到** $T(n) = \Theta(f(n)) = n \log n$.



主方法 (公式法)

上述三类情况并没有覆盖所有可能的 $f(n)$, 每类之间存在空隙.

- 给定 $w_1 = n^{\log_b a}$, $w_2 = n^{-\varepsilon}$, $w_3 = n^{\varepsilon}$.
- 第一种情况: $f(n)$ 不仅必须比 w_1 小，而且必须是多项式地比 w_1 小，即 $f(n)$ 必须渐近地小于 $w_1 \cdot w_2$.
- 第三种情况: $f(n)$ 不仅必须比 w_1 大, 而且必须是多项式地比 w_1 大，还需满足附加的“正规性”条件，对常数 $c < 1$ 与所有足够大的 n, $af(n/b) \leq cf(n)$.



主方法 (公式法)

$T(n) = 2T(n/2) + n \log n$

- $n^{\log_b a} = n$, $f(n) = n \log n$.
- 虽然 $f(n)$ 渐近地大于 n, 但 $f(n)$ 并不是多项式地大于 n,

$$\frac{f(n)}{n^{1+\varepsilon}} = \frac{n \log n}{n^{1+\varepsilon}} = n^{-\varepsilon} \log n \to 0.$$



主方法 (公式法)

$T(n) = 2T(n/2) + n \log n$

- $n^{\log_b a} = n$, $f(n) = n \log n$.
- 虽然 $f(n)$ 渐近地大于 n, 但 $f(n)$ 并不是多项式地大于 n,

$$\frac{f(n)}{n^{1+\varepsilon}} = \frac{n \log n}{n^{1+\varepsilon}} = n^{-\varepsilon} \log n \to 0.$$

$f(x)$ 多项式地大于 $g(x)$: 存在实数 $\varepsilon > 0$, 使得

$$f(x) > g(x) n^{\varepsilon}$$

即两个函数的商大于一个多项式.

差分方程法

有些递归方程可以看成一个差分方程，因而可以用解差分方程 (初值问题) 的方法来解递归方程。然后对得到的解作渐近阶的估计。



生成函数法 (母函数法)

这是一个有广泛适用性的方法。它不仅可以用来求解线性常系数高阶齐次和非齐次的递归方程，而且可以用来求解线性变系数高阶齐次和非齐次的递归方程，甚至可以用来求解非线性递归方程。方法的基本思想是设定递归方程解的母函数，努力建立一个关于母函数的可解方程，将其解出，然后返回递归方程的解。

